

Mini Project Kalkulator BMI dengan Python dan Streamlit

Tentang Streamlit

Streamlit adalah *framework* Python yang memungkinkan kita mengubah script Python menjadi aplikasi web interaktif dalam hitungan menit. Kita tidak perlu menguasai HTML, CSS, atau JavaScript untuk membuat antarmuka yang menarik bagi proyek data atau alat bantu sederhana lainnya.

Untuk mempelajarinya secara bertahap, kita akan membuat mini project berupa **Kalkulator BMI (Body Mass Index)**. Proyek ini akan membantu kita memahami bagaimana Streamlit menangani input dari pengguna, melakukan perhitungan di balik layar, dan menampilkan hasilnya kembali ke layar secara *real-time*.

Yang akan kita pelajari adalah:

1. **Persiapan & Instalasi:** Cara membuat virtual environment dan memasang Streamlit di komputer serta menjalankan script "Hello World" pertama kita.
2. **Membangun Antarmuka (UI):** Mengetahui cara membuat *header*, kolom input angka, tombol, dan sidebar untuk menerima data dari pengguna.
3. **Logika Kalkulator & Output:** Menghubungkan input dengan rumus matematika dan menampilkan hasil serta kategori kesehatan.
4. **Refactoring:** Memecah kode menjadi beberapa file agar lebih mudah dipelihara dan diuji.

Instalasi Streamlit

Panduan ini akan memandu Anda membuat lingkungan dengan venv dan menginstal Streamlit dengan pip yang direkomendasikan. Setelah streamlit terinstall, kita akan membuat aplikasi "Hello world" sederhana dan menjalankannya.

Prasyarat

- Pastikan komputer Anda telah terinstall Python dengan versi yang sesuai (3.10 s.d. 3.14).
- Python package manager (pip)
- Code editor (VSCode, dll)

Untuk menyiapkan lingkungan Python dan menguji instalasi, jalankan perintah terminal berikut:

```
mkdir myproject
cd myproject
python --version
python -m venv .venv

# Windows command prompt
.venv\Scripts\activate.bat

# Windows PowerShell
.venv\Scripts\Activate.ps1
```

```
# macOS and Linux
source .venv/bin/activate

pip install streamlit
streamlit hello
```

Setelah menjalankan perintah `streamlit hello` di terminal, browser akan terbuka secara otomatis dan menampilkan halaman sambutan interaktif yang berisi berbagai demo fitur.

Struktur Proyek

Proyek Kalkulator BMI yang kita bangun memiliki struktur sebagai berikut:

```
kalkulator-bmi/
├── app.py                # Titik masuk utama aplikasi
├── requirements.txt      # Daftar dependencies
├── logic/
│   └── bmi.py           # Logika perhitungan BMI (fungsi murni)
├── components/
│   ├── sidebar.py       # Komponen input di sidebar
│   └── results.py       # Komponen tampilan hasil
```

Pemisahan ini mengikuti prinsip **Separation of Concerns**, yaitu memisahkan logika program dari antarmuka pengguna agar kode lebih mudah diuji dan dipelihara.

Buat Script Kalkulator Sederhana

Langkah selanjutnya adalah membuat file script Python untuk aplikasi Kalkulator BMI. Di tahap ini, kita akan fokus membangun tampilan agar pengguna bisa memasukkan data.

Silakan buat file baru bernama `app.py` di folder kerja, lalu masukkan kode berikut:

```
# app.py
import streamlit as st

# Menampilkan judul aplikasi
st.title("Kalkulator BMI Interaktif")

# Membuat input nama pengguna
nama = st.text_input("Masukkan nama Anda:")

# Membuat input jenis kelamin pengguna
jenis_kelamin = st.selectbox("Pilih jenis kelamin Anda:", ["Laki-laki", "Perempuan"])

# Membuat input angka untuk berat badan
berat = st.number_input("Masukkan berat badan Anda (kg):", min_value=1.0, step=0.1)
```

```
# Membuat input angka untuk tinggi badan
tinggi = st.number_input("Masukkan tinggi badan Anda (cm):", min_value=1.0,
step=1.0)
```

Setelah menyimpan file tersebut, buka terminal di folder yang sama dan ketik perintah berikut:

```
streamlit run app.py
```

Setelah menjalankan perintah di atas, tab baru akan terbuka di browser dan menampilkan kotak-kotak input untuk nama, jenis kelamin, berat badan, dan tinggi badan.

Penjelasan Komponen UI

Komponen	Fungsi
<code>st.title()</code>	Menampilkan judul besar di bagian atas aplikasi
<code>st.text_input()</code>	Membuat kotak input untuk teks (nama)
<code>st.selectbox()</code>	Membuat dropdown untuk memilih salah satu opsi (jenis kelamin)
<code>st.number_input()</code>	Membuat kotak input khusus angka. Parameter <code>min_value</code> memastikan pengguna tidak memasukkan angka nol atau negatif

Tambahkan Logika Perhitungan BMI

BMI dihitung dengan membagi berat badan (kg) dengan kuadrat tinggi badan dalam satuan meter:

$$\text{BMI} = \text{berat (kg)} / \text{tinggi (m)}^2$$

Karena pengguna memasukkan tinggi dalam satuan sentimeter (cm), maka kita harus mengkonversi variabel tinggi ke dalam meter terlebih dahulu (`tinggi_m = tinggi / 100`), baru kemudian menghitung BMI-nya.

```
# 1. Konversi tinggi dari cm ke meter
tinggi_m = tinggi / 100

# 2. Hitung BMI (berat dibagi tinggi pangkat dua)
# Di Python, pangkat dua ditulis dengan simbol **2
bmi = berat / (tinggi_m ** 2)
```

Dengan kode di atas, setiap kali pengguna mengubah angka di aplikasi web, Streamlit akan otomatis menjalankan ulang script dan menghitung nilai BMI yang baru.

Sekarang kita sudah punya nilai BMI yang tersimpan di variabel `bmi`. Namun, nilai ini baru ada di "ingatan" komputer dan belum muncul di layar aplikasi web kita. Untuk menampilkan hasil di layar, kita bisa

menggunakan `st.write` atau `st.success`.

Berikut cara kita menulisnya dalam kode:

```
# Menampilkan hasil dengan 2 angka di belakang koma agar rapi
st.write(f"Nilai BMI Anda adalah: {bmi:.2f}")

# Atau jika ingin lebih cantik dengan warna hijau:
st.success(f"Hasil perhitungan BMI: {bmi:.2f}")
```

Simbol `:.2f` di dalam kode tersebut berfungsi untuk membatasi angka desimal agar tidak terlalu panjang (misalnya dari `22.4489...` menjadi `22.45`).

Sekarang, angka BMI sudah muncul di layar. Namun, pengguna mungkin tidak tahu apakah angka 22.5 itu termasuk sehat atau tidak.

Dalam Python, kita biasanya menggunakan `if-elif-else` untuk mengecek rentang nilai (seperti nilai BMI). Berikut adalah tabel kategori BMI secara umum:

Rentang BMI	Kategori BMI
Kurang dari 18.5	Berat Badan Kurang (Underweight)
18.5 – 24.9	Berat Badan Ideal (Normal)
25.0 – 29.9	Berat Badan Berlebih (Overweight)
30.0 atau lebih	Obesitas

Struktur `if-elif-else` di Python bekerja seperti percabangan jalan. Program akan memeriksa satu per satu kondisi dari atas ke bawah, dan berhenti begitu menemukan kondisi yang benar (True).

Berikut adalah cara kita menuliskan logika kategori BMI tersebut ke dalam kode:

```
if bmi < 18.5:
    kategori = "Berat Badan Kurang (Underweight)"
    st.warning(kategori)
elif 18.5 <= bmi < 24.9:
    kategori = "Berat Badan Ideal (Normal)"
    st.success(kategori)
elif 25.0 <= bmi < 29.9:
    kategori = "Berat Badan Berlebih (Overweight)"
    st.warning(kategori)
else:
    kategori = "Obesitas"
    st.error(kategori)
```

Penjelasan Kode

- `if`: Kondisi pertama yang diperiksa.

- **elif** (else if): Digunakan untuk memeriksa kondisi berikutnya jika kondisi sebelumnya salah. Kita bisa menggunakan **elif** sebanyak yang dibutuhkan.
- **else**: Pilihan terakhir jika tidak ada satupun kondisi di atas yang terpenuhi (dalam hal ini, jika BMI 30 atau lebih).
- **st.warning** / **st.success** / **st.error**: Cara Streamlit menampilkan pesan dengan warna berbeda (kuning, hijau, dan merah) agar tampilan lebih informatif.

Menampilkan Hasil dengan Fungsi Terpisah

Untuk membuat tampilan hasil yang lebih rapi dan terorganisir, kita bisa memindahkan kode tampilan hasil ke dalam sebuah fungsi tersendiri.

```
def tampilkan_hasil_bmi(nama, gender, nilai_bmi, kategori, saran):  
    """Fungsi khusus untuk mengurus tampilan (render) hasil BMI ke layar utama."""  
    st.header("🏠 Hasil Analisis Kesehatan")  
    st.subheader(f"Halo, {nama}!")  
    st.write(f"Jenis Kelamin: {gender}")  
    st.divider()  
    # Menampilkan metrik utama  
    st.metric(label="Nilai BMI Anda", value=f"{nilai_bmi:.2f}")  
    st.info(f"Kategori BMI: **{kategori}**\n\n**Saran**: {saran}")
```

Penjelasan Komponen Baru

Komponen	Fungsi
<code>st.header()</code>	Menampilkan judul bagian (lebih kecil dari <code>st.title</code>)
<code>st.subheader()</code>	Menampilkan sub-judul untuk memberi konteks
<code>st.metric()</code>	Menampilkan angka dengan format yang lebih menonjol dan profesional
<code>st.divider()</code>	Membuat garis horizontal untuk merapikan tampilan teks
<code>st.info()</code>	Menampilkan kotak informasi dengan warna biru

Menggunakan Sidebar

Menggunakan sidebar adalah cara efektif untuk memisahkan antara input/konfigurasi dengan hasil/konten utama. Di Streamlit, kita bisa memindahkan hampir semua elemen ke samping hanya dengan menambahkan kata `.sidebar` setelah `st`.

Bayangkan halaman web kita memiliki dua area:

- **Main Area**: Tempat untuk judul besar, grafik, atau hasil.
- **Sidebar**: Area di sebelah kiri (atau kanan) untuk kontrol atau input data.

Jika sebelumnya kita menulis:

```
nama = st.text_input("Masukkan nama Anda:")
```

Maka untuk memindahkannya ke samping kiri, kodenya menjadi:

```
nama = st.sidebar.text_input("Masukkan nama Anda:")
```

Berikut contoh lengkap bagaimana kita membuat input di sidebar:

```
def render_sidebar():
    """Merender komponen input di sidebar dan mengembalikan data
    pengguna."""
    st.sidebar.title("Silahkan isi data berikut")

    nama = st.sidebar.text_input("Nama:", placeholder="Masukkan nama Anda")
    jenis_kelamin = st.sidebar.radio("Jenis Kelamin:", ["Laki-laki",
    "Perempuan"])

    # Memberikan nilai default (value) yang logis agar aplikasi tidak
    langsung menghitung angka 1.0
    berat = st.sidebar.number_input("Masukkan Berat Badan (kg)",
                                     min_value=1.0, value=60.0, step=0.1)
    tinggi = st.sidebar.number_input("Masukkan Tinggi Badan (cm)",
                                     min_value=1.0, value=165.0, step=0.1)

    tombol_hitung = st.sidebar.button("Hitung BMI", type="primary")

    st.sidebar.divider()
    st.sidebar.markdown("""
    ### Tentang BMI 💡
    **Body Mass Index (BMI)** adalah cara sederhana untuk memantau status
    gizi orang dewasa.

    * **Penting:** Skor ini tidak memperhitungkan masa otot, kepadatan
    tulang, dan komposisi tubuh secara keseluruhan.
    """)
    return nama, jenis_kelamin, berat, tinggi, tombol_hitung
```

Penjelasan Komponen Sidebar

Komponen	Fungsi
<code>st.sidebar.title()</code>	Memberikan judul di area sidebar
<code>st.sidebar.radio()</code>	Membuat pilihan berbentuk radio button (hanya bisa pilih satu)
<code>st.sidebar.button()</code>	Membuat tombol yang bisa diklik. Parameter <code>type="primary"</code> membuat tombol terlihat lebih menonjol

Komponen	Fungsi
<code>st.sidebar.divider()</code>	Garis pemisah di sidebar
<code>st.sidebar.markdown()</code>	Menampilkan teks berformat markdown di sidebar

Tambahkan Progress Bar BMI

Menambahkan Progress Bar akan memberikan visualisasi instan mengenai posisi berat badan seseorang dalam skala kesehatan secara lebih intuitif.

Di Streamlit, kita bisa menggunakan fungsi `st.progress()`. Namun, ada satu aturan teknis yang penting: nilai yang dimasukkan ke dalam fungsi ini harus berupa angka desimal antara `0.0` (bar kosong) hingga `1.0` (bar penuh).

Masalahnya, angka BMI kita biasanya berkisar antara 10 hingga 40 atau lebih. Kita tidak bisa langsung memasukkan variabel `bmi` ke dalam progress bar karena nilainya pasti lebih besar dari `1.0`. Jadi, kita perlu melakukan sedikit perhitungan untuk memetakan angka BMI tersebut ke dalam skala 0 hingga 1.

Misalnya, kita tetapkan bahwa angka BMI 40 adalah batas maksimal (artinya bar akan terisi penuh atau `1.0`):

```
nilai_aman = min(bmi / 40, 1.0)
st.progress(nilai_aman)
```

Fungsi `min(angka1, angka2)` akan mengambil angka yang paling kecil di antara keduanya. Ini memastikan bahwa jika ada pengguna dengan BMI lebih dari 40 (misalnya 45), maka nilai yang dimasukkan ke `st.progress()` tetap maksimal `1.0`, sehingga aplikasi tidak mengalami error.

Refactoring: Memecah Kode Menjadi Beberapa File

Seiring berkembangnya fitur aplikasi, kode dalam satu file (`app.py`) bisa menjadi sangat panjang dan sulit dikelola. Oleh karena itu, kita perlu melakukan **refactoring**, yaitu mengubah struktur kode tanpa mengubah fungsinya agar lebih mudah dibaca dan dipelihara.

Masalah yang Dihadapi

Pada kode awal, kita meletakkan semua logika — mulai dari input, perhitungan, pencategorian, hingga tampilan — dalam satu file. Ini memiliki beberapa kekurangan:

1. **Sulit diuji:** Tidak bisa menguji fungsi logika perhitungan secara independen.
2. **Kode berantakan:** Semakin banyak fitur, semakin panjang file-nya.
3. **Violasi Separation of Concerns:** Logika bisnis bercampur dengan tampilan UI.

Solusi: Pemisahan Modul

Kita memecah `app.py` menjadi beberapa file dengan tanggung jawab masing-masing:

```

kalkulator-bmi/
├── app.py                # Titik masuk utama
├── logic/
│   └── bmi.py            # Logika perhitungan BMI (fungsi murni)
└── components/
    ├── sidebar.py        # Komponen input di sidebar
    └── results.py        # Komponen tampilan hasil

```

File Logic: `logic/bmi.py`

File ini berisi **fungsi murni** (pure function) yang hanya bertanggung jawab untuk menghitung BMI dan menentukan kategorinya. Fungsi ini tidak bergantung pada komponen Streamlit sehingga mudah diuji.

```

"""Pure function for BMI calculation and categorization."""

def hitung_dan_kategorikan_bmi(berat_kg: float, tinggi_cm: float) ->
tuple[float, str, str]:
    """
    Fungsi murni untuk menghitung skor BMI,
    menentukan kategori, dan memberikan saran kesehatan.
    """
    tinggi_m = tinggi_cm / 100
    skor_bmi = berat_kg / (tinggi_m ** 2)

    if skor_bmi < 18.5:
        kategori = "Berat Badan Kurang (Underweight)"
        saran = "Perbanyak asupan nutrisi dan kalori sehat."
    elif 18.5 <= skor_bmi < 24.9:
        kategori = "Berat Badan Ideal (Normal)"
        saran = "Pertahankan pola makan seimbang dan olahraga rutin."
    elif 25.0 <= skor_bmi < 29.9:
        kategori = "Berat Badan Berlebih (Overweight)"
        saran = "Cobalah untuk lebih aktif bergerak dan kurangi gula."
    else:
        kategori = "Obesitas"
        saran = "Disarankan berkonsultasi dengan tenaga medis."

    return skor_bmi, kategori, saran

```

File Components: `components/sidebar.py`

File ini berisi fungsi untuk merender komponen input di sidebar. Fungsi `render_sidebar()` mengembalikan semua data yang dibutuhkan dari pengguna.

```

"""Sidebar components for user input."""

def render_sidebar():
    """Render sidebar input components and return user data."""

```



```

import streamlit as st

st.sidebar.title("Silahkan isi data berikut")

nama = st.sidebar.text_input("Nama:", placeholder="Masukkan nama Anda")
jenis_kelamin = st.sidebar.radio("Jenis Kelamin:", ["Laki-laki",
"Perempuan"])

berat = st.sidebar.number_input("Masukkan Berat Badan (kg)",
                                min_value=1.0, value=60.0, step=0.1)
tinggi = st.sidebar.number_input("Masukkan Tinggi Badan (cm)",
                                min_value=1.0, value=165.0, step=0.1)

tombol_hitung = st.sidebar.button("Hitung BMI", type="primary")

st.sidebar.divider()
st.sidebar.markdown("""
### Tentang BMI 💡
**Body Mass Index (BMI)** adalah cara sederhana untuk memantau status
gizi orang dewasa.

*   **Penting:** Skor ini tidak memperhitungkan masa otot, kepadatan
tulang, dan komposisi tubuh secara keseluruhan.
""")
return nama, jenis_kelamin, berat, tinggi, tombol_hitung

```

File Components: `components/results.py`

File ini berisi fungsi untuk menampilkan hasil perhitungan BMI di area utama aplikasi.

```

"""Results display components."""

def tampilkan_hasil_bmi(nama_pengguna, gender, nilai_bmi, kategori_bmi,
saran_bmi):
    """Fungsi khusus untuk mengurus tampilan hasil BMI ke layar utama."""
    import streamlit as st

    st.header("🏠 Hasil Analisis Kesehatan")
    st.subheader(f"Halo, {nama_pengguna}!")
    st.write(f"Jenis Kelamin: {gender}")
    st.divider()
    st.metric(label="Nilai BMI Anda", value=f"{nilai_bmi:.2f}")
    st.info(f"Kategori BMI: **{kategori_bmi}**\n\n**Saran**:: {saran_bmi}")

    st.write("Posisi BMI Anda dalam skala umum (maksimal skala 40):")
    progress_bmi = min(nilai_bmi / 40, 1.0)
    st.progress(progress_bmi)

def tampilkan_welcome():
    """Display welcome page when no calculation is triggered."""

```

```
import streamlit as st

st.title("🏃 Kalkulator BMI")
st.markdown("""
### Selamat Datang!
Silakan isi data diri pada **panel di sebelah kiri**, lalu klik tombol
**Hitung BMI** untuk mengetahui skor BMI dan kategori kesehatan Anda.
""")
```

File Utama: **app.py**

File ini sekarang menjadi sangat rapi dan hanya berisi orchestrasi alur aplikasi.

```
"""BMI Calculator Application – Refactored."""
import streamlit as st

from logic.bmi import hitung_dan_kategorikan_bmi
from components.sidebar import render_sidebar
from components.results import tampilkan_hasil_bmi, tampilkan_welcome

# =====
# MAIN APPLICATION
# =====

# Render sidebar and get user inputs
nama, jenis_kelamin, berat, tinggi, tombol_hitung = render_sidebar()

# Jika tombol diklik, jalankan validasi dan tampilkan hasil
if tombol_hitung:
    if len(nama.strip()) < 3:
        st.sidebar.error("⚠️ Nama minimal terdiri dari 3 karakter (spasi tidak dihitung).")
    else:
        # Panggil fungsi logika
        skor_bmi, kategori, saran = hitung_dan_kategorikan_bmi(berat, tinggi)
        # Tampilkan hasil di area utama
        tampilkan_hasil_bmi(nama, jenis_kelamin, skor_bmi, kategori, saran)

# Jika tombol belum diklik, tampilkan halaman Welcome
else:
    tampilkan_welcome()
```

Manfaat Refactoring

1. **Separation of Concerns:** Logika bisnis terpisah dari UI, sehingga mudah dimodifikasi tanpa saling mengganggu.
2. **Mudah Diuji:** Setiap fungsi bisa diuji secara independen.
3. **Kode Bersih:** File utama (**app.py**) menjadi sangat ringkas dan mudah dibaca.

4. **Dapat Digunakan Kembali:** Komponen di `components/` bisa dipakai ulang di proyek lain.